

SV-LIB 1.0

A Standard Exchange Format for Software-Verification Tasks

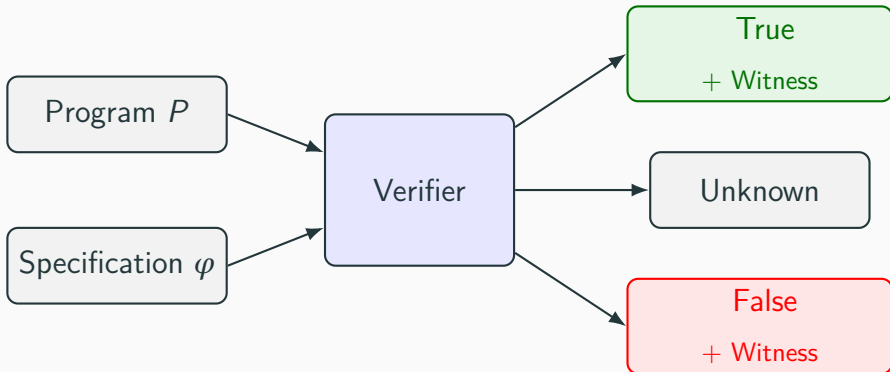
Marian Lingsch-Rosenfeld 

Dirk Beyer  Gidon Ernst  Martin Jonáš 

2026-09-23
LMU Munich, Germany



Software Verification



Decide whether a program satisfies its specification

Software System

```
int sqliteProcessJoin(Parse *pParse, Select *p){
    ist *pSrc;           /* All tables in the FROM clause */
    i, j;                /* Loop counters */
    ct SrcList_item *pLeft; /* Left table being joined */
    ct SrcList_item *pRight; /* Right table being joined */

    = p->pSrc;
    t = &pSrc->a[0];
    nt = &pLeft[1];
    i=0; i<pSrc->nSrc-1; i++, pRight++, pLeft++){
        ble *pRightTab = pRight->pTab;
        t isOuter;
```

```
{ NEVER(pLeft->pTab==0 || pRightTab==0) } continue;
Outer = (pRight->fg.jointype & JT_OUTER)!=0;
```

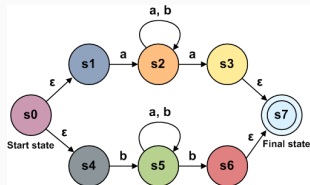
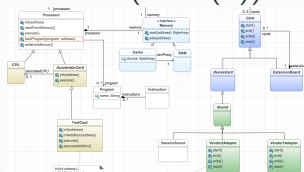
When the NATURAL keyword is present, add WHERE clause terms for every column that the two tables have in common.

```
{ pRight->fg.jointype & JT_NATURAL ){
    if( pRight->pOn || pRight->pUsing ){
        sqlite3ErrorMsg(pParse, "a NATURAL join may not have "
            "an ON or USING clause", 0);
        return 1;
    }
    for(j=0; j<pRightTab->nCol; j++){
        char *zName; /* Name of column in the right table */
        int ileft; /* Matching left table */
        int ileftCol; /* Matching column in the left table */

        zName = pRightTab->aCol[j].zName;
        if( tableAndColumnIndex(pSrc, i+1, zName, &ileft, &ileftCol)
```

Abstract Model

$$\forall x. \text{read}(\text{write}(x)) = x$$



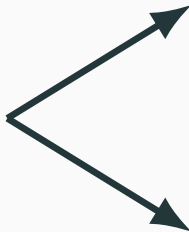
Which Model?

```
int x = nondet();  
int y = x + 1;  
assert(y > x);
```

Is the program safe?

Which Model?

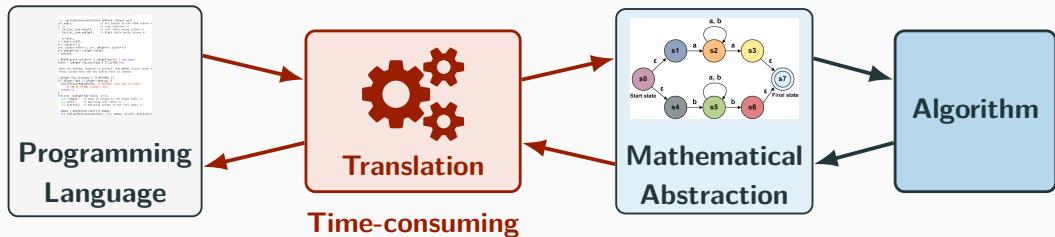
```
int x = nondet();  
int y = x + 1;  
assert(y > x);
```



```
; Integer Encoding  
(declare-const x Int)  
(declare-const y Int)  
(assert (= y (+ x 1)))  
(assert (not (> y x)))  
(check-sat) ; unsat
```

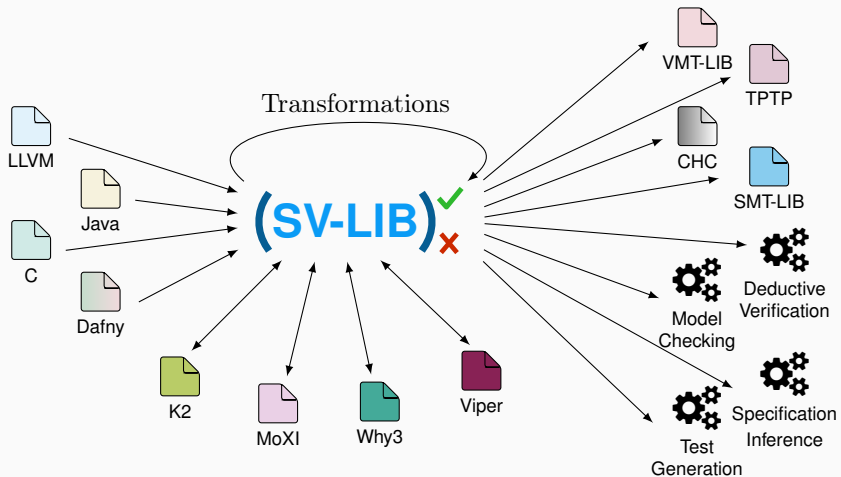
```
; Bitvector Encoding  
(declare-const x (_ BitVec 32))  
(declare-const y (_ BitVec 32))  
(assert (= y (bvadd x (_ bv1 32))))  
(assert (not (bvsgt y x)))  
(check-sat) ; sat, at INT_MAX
```

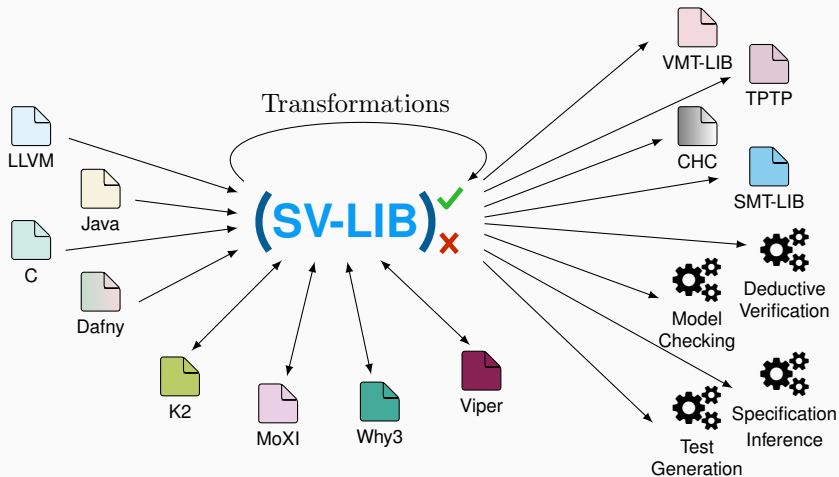
Verification Process



Building the translation is time-consuming!

SV-LIB





(a) Inspired by k2 [14]

(b) Based on SMT-LIB [2]

Motivation for another IVL

- (a) Reduce engineering complexity (e.g., easy to parse and interpret)
- (b) Complexities of the input language transfer to complexity in presenting verification results (e.g., witnesses)
- (c) Information exchange between tools is difficult (e.g., witnesses, ...)
- (d) Have formal semantics without undefined behavior
- (e) many more! [5]

SV-LIB: Goals

- (a) Consolidate existing IVLs from an engineering perspective by taking well-understood constructs, and avoiding semantic complexity
- (b) Keep programs and specifications separable and the latter extensible
- (c) Make witnesses first-class citizens by providing a standard format for them
- (d) Make implementation of verification tools easier
- (e) Strive for a software ecosystem by providing building blocks (parser, linter, default validator, transformations, benchmarks, . . .)

SV-LIB

Modelling a Problem: SMT-LIB

5	3			7				
6			1	9	5			
	9	8					6	
8				6				3
4			8		3			1
7				2				6
	6					2	8	
			4	1	9			5
				8			7	9

Every row, every column and every 3×3 box holds the digits 1 to 9, each exactly once.

Which digits go into the empty cells?

To let a solver answer this, we first have to model this.

Modelling Sudoku with SMT-LIB

```
(set-logic QF_LIA)
; one unknown per cell, by row and column
(declare-const x11 Int) ... (declare-const x99 Int)
; every cell holds a digit
(assert (and (<= 1 x11 9) ... (<= 1 x99 9)))
; the digits the puzzle gives us
(assert (and (= x11 5) (= x12 3) (= x15 7)))
; row, column and box hold different digits
(assert (distinct x11 x12 x13 x14 x15 x16 x17 x18 x19))
(assert (distinct x11 x21 x31 x41 x51 x61 x71 x81 x91))
(assert (distinct x11 x12 x13 x21 x22 x23 x31 x32 x33))
; the query, and the request for a solution
(check-sat)
(get-model)
```

5	3	4	6	7	8	9	1	2
6	7	2	1	9	5	3	4	8
1	9	8	3	4	2	5	6	7
8	5	9	7	6	1	4	2	3
4	2	6	8	5	3	7	9	1
7	1	3	9	2	4	8	5	6
9	6	1	5	3	7	2	8	4
2	8	7	4	1	9	6	3	5
3	4	5	2	8	6	1	7	9

bash

```
z3 sudoku.smt2 # sat -- and the model is the solved grid
```

From SMT-LIB to SV-LIB

What SMT-LIB gives us

One syntax for sorts, terms and assertions

`sat` / `unsat` / `unknown` as the verdict

A model as the *witness* for `sat`

Solvers are interchangeable

CHCs fit here as they model a program
through clauses, written in SMT-LIB

From SMT-LIB to SV-LIB

What SMT-LIB gives us

One syntax for sorts, terms and assertions

`sat` / `unsat` / `unknown` as the verdict

A model as the *witness* for `sat`

Solvers are interchangeable

CHCs fit here as they model a program through clauses, written in SMT-LIB

What a program still needs

Variables that change: `declare-var`

Procedures with a body: `define-proc`

Statements: assignments, loops, calls, `gotos`

Properties at program points: `annotate-tag`

A query about a call: `verify-call`

A witness for *both* answers: `get-witness`

A verification task is an SMT-LIB script that also talks about programs

A Verification Task

```
1 (set-logic LIA)
2
3 (define-proc
4   add
5   ((x0 Int) (y0 Int))
6   ((x Int))
7   ((y Int))
8   (! (sequence
9       (assign (x x0) (y y0))
10      (! (while
11          (< 0 y)
12          (assign
13            (x (+ x 1))
14            (y (- y 1))))
15        :tag while-loop))
16     :tag proc-add))
```

```
17 (annotate-tag
18   proc-add
19   :requires (<= 0 y0)
20   :ensures (= x (+ x0 y0)))
21
22 (annotate-tag while-loop :not-recurring)
23
24 (declare-const x1 Int)
25 (declare-const y1 Int)
26 (assert (<= 0 y1))
27
28 (verify-call add (x1 y1))
```

add counts y down and x up, so it returns $x_0 + y_0$

Commands

```
; from SMT-LIB: sorts, terms, symbols, and a subset of its commands
(set-logic LIA) (declare-const x1 Int) (define-fun f () Int 0)
; a global program variable, which may be assigned to
(declare-var x Int)
; a procedure: name, inputs, outputs, locals, body
(define-proc add ((x0 Int) (y0 Int)) ((x Int)) ((y Int)) stmt)
(define-procs-rec (signature)n+1 (stmt)n+1)
; the specification: properties attached to a tag
(annotate-tag proc-add :requires (<= 0 y0)
                      :ensures  (= x (+ x0 y0)))
; the query, and the request for a witness
(verify-call add (x1 y1))
(get-witness)
```

Statements

```
; basic
(assume  $t$ )  (assign  $(x\ t)^+$ )  (sequence  $s^*$ )  (!  $s\ attr^+$ )
; procedures
(call  $p\ (t^*)\ (x^*)$ )  (return)
; unstructured control flow
(label  $l$ )  (goto  $l$ )  (if  $t$  (goto  $l$ ))
; structured control flow
(if  $t\ s\ s^?$ )  (while  $t\ s$ )  (break)  (continue)
; nondeterminism
(havoc  $x^+$ )  (choice  $s^+$ )
```

Properties

Attribute	Attached to	Meaning
<i>Safety</i>		
:check-true t	statement	t holds whenever the statement is reached
:requires t	procedure	precondition
:ensures t	procedure	postcondition
:invariant t	loop	loop invariant
<i>Liveness</i>		
:recurring	statement	reached infinitely often
:not-recurring	statement	reached only finitely often
:decreases t	loop	ranking function
:decreases-lex (t^+)	loop	lexicographic ranking function

Extensible as new properties do not change the program

Correctness Witnesses in SV-LIB

```
1 (set-logic LIA)
2
3 (define-proc
4   add
5   ((x0 Int) (y0 Int))
6   ((x Int))
7   ((y Int))
8   (! (sequence
9       (assign (x x0) (y y0))
10      (! (while
11          (< 0 y)
12          (assign
13            (x (+ x 1))
14            (y (- y 1))))
15        :tag while-loop))
16     :tag proc-add))
```

```
17 (annotate-tag
18   proc-add
19   :requires (<= 0 y0)
20   :ensures (= x (+ x0 y0)))
21
22 (annotate-tag while-loop :not-recurring)
23
24 (declare-const x1 Int)
25 (declare-const y1 Int)
26 (assert (<= 0 y1))
27
28 (verify-call add (x1 y1))
```

Correctness Witness

```
((annotate-tag while-loop
  :invariant
    (and (<= 0 y) (= (+ x y) (+ x0 y0)))
  :decreases y))
```

Violation Witnesses in SV-LIB

```
1 (set-logic LIA)
2
3 (define-proc
4   add
5   ((x0 Int) (y0 Int))
6   ((x Int))
7   ((y Int))
8   (! (sequence
9       (assign (x x0) (y y0))
10      (! (while
11          (<= 0 y)
12          (assign
13              (x (+ x 1))
14              (y (- y 1))))
15          :tag while-loop))
16      :tag proc-add))
17 [...]
```

Violation Witness

```
((select-trace
  (model
    (define-fun x1 () Int 1)
    (define-fun y1 () Int 1))
  (init-global-vars)
  (entry-proc add)
  (steps
    (init-proc-vars add
      ; initialization of x and y
      ; not necessary
    ))
  (incorrect-annotation
    proc-add
    :ensures
    (= x (+ x0 y0)))))
```

Validation is Verification

```
1 (set-logic LIA)
2
3 (define-proc
4   add
5   ((x0 Int) (y0 Int))
6   ((x Int))
7   ((y Int))
8   (! (sequence
9       (assign (x x0) (y y0))
10      (! (while
11          (< 0 y)
12          (assign
13            (x (+ x 1))
14            (y (- y 1))))
15        :tag while-loop))
16     :tag proc-add))
```

```
17 (annotate-tag
18   proc-add
19   :requires (<= 0 y0)
20   :ensures (= x (+ x0 y0))
21   :not-recurring)
22
23 (declare-const x1 Int)
24 (declare-const y1 Int)
25 (assert (<= 0 y1))
26
27 ; Command taken from the witness
28 (annotate-tag while-loop
29   :invariant
30   (and (<= 0 y) (= (+ x y) (+ x0 y0)))
31   :decreases y)
32
33 (verify-call add (x1 y1))
```

SV-LIB Ecosystem

Goal: Software Ecosystem

Aim to provide building blocks to facilitate adoption:

- (a) [ANTLR grammar for parsing SV-LIB](#) [5]
- (b) [PYSVLIB: Python library and command-line tool](#) [5]
- (c) [Integration of SV-LIB into CPACHECKER](#) [1]
- (d) [Lightweight model checker SVLIBCHECKER](#) [6]
- (e) [Translations into SV-LIB: Btor2, MoXI, LLVM](#)
- (f) [Backends out of SV-LIB: CHC](#)

Every item links to where it lives

Follow Along

Everything on the next slides is in the demo repository



<https://gitlab.com/sosy-lab/research/data/sv-lib-demo>

Setup

bash

```
git clone https://gitlab.com/sosy-lab/research/data/sv-lib-demo
cd sv-lib-demo/tutorial
python3 -m venv venv && source venv/bin/activate

# PySvLib, which brings the 'pysvlib' command
pip install "git+https://gitlab.com/sosy-lab/benchmarking/sv-lib.git\
@b739ef8a36a09aeda4df771471f5f144051a4e4a#subdirectory=pysvlib"

# SMT solvers and CPAChecker; the demo also lists source builds
apt install z3 cvc5 cpachecker # and container images for each

# SvLibChecker: pre-built package from CI, or built from source
git clone https://gitlab.com/sosy-lab/software/svlibchecker.git
cd svlibchecker && git checkout dc0c6699 && make package
```

bash

```
cd smtlib

# the Sudoku, and the same file for another solver
z3 sudoku.smt2 # sat, and the model is the solved grid
cvc5 sudoku.smt2 # sat, with the same model

# the same puzzle, modelled with arrays and quantifiers
z3 -T:10 sudoku-arrays.smt2 # timeout -- the model has its price

# the same program, two models of 'int'
z3 integer-encoding.smt2 # unsat -- the assertion cannot fail
z3 bitvector-encoding.smt2 # sat -- x = #x7fffffff
```

The encoding is a choice which is made explicit in SV-LIB

The Task

```
1 (set-logic LIA)
2 (set-option :produce-witnesses true)
3 (set-info :format-version "1.0")
4 (define-proc
5   main
6   ()
7   ((|c#result| Int))
8   ((a Int))
9   (! (sequence
10      (assign (a 0))
11      (! (while
12         (< a 6)
13         (assign (a (+ a 1)))) :tag while-loop)
14      (if (not (= a 6)) (! (sequence) :tag assertion :check-true false))
15      (assign (|c#result| 1))
16      (return)) :tag proc-main))
17 (verify-call main ())
18 (get-witness)
```

loop-simple-safe.svlib

A loop counting up to 6, whose assertion holds.

loop-simple-unsafe.svlib is the same program with a property that does not hold.

PySvLib: Reading and Checking Files

bash

```
pysvlib lint loop-simple-safe.svlib
```

```
# done
```

```
pysvlib lint --print-file-metadata loop-simple-safe.svlib
```

```
# Linting Verification task: loop-simple-safe.svlib
```

```
# Verification task File hash: 65186ceba6e39b417776f77f9957967a21a8...
```

```
# Verification task File lines of code: 30
```

```
# done
```

One default python parser and linter is provided

PySvLib: Verification and Validation

bash

```
# encode the task into Constrained Horn Clauses and solve them
```

```
pysvlib chc --solver z3 loop-simple-safe.svlib # correct
```

```
pysvlib chc --solver z3 loop-simple-unsafe.svlib # incorrect
```

```
# the solution found is exported as a correctness witness
```

```
cat output/witness.svlib
```

```
# validating a witness is running a validator on the linked task
```

```
pysvlib validate --solver z3 --witness output/witness.svlib \  
loop-simple-safe.svlib # correct
```

CPAchecker [1]

bash

```
cpachecker loop-simple-safe.svlib  
# Verification result: TRUE. No property violation found [...]  
cpachecker loop-simple-unsafe.svlib  
# Verification result: FALSE. Property violation [...]
```

The input language is detected from the file, and output/witness.svlib holds the argument for why the program is safe:

Correctness Witness

```
(annotate-tag while-loop :invariant (or (and (not (= a 6)) (<= a 0)) ...))  
(annotate-tag proc-main :requires true)  
(annotate-tag proc-main :ensures (=> true true))
```

SvLibChecker [6]

bash

```
# one configuration per algorithm, on the very same task
./svlibchecker/svlibchecker.py --config predicateAnalysisCegar \
  loop-simple-safe.svlib # Verification result: correct
./svlibchecker/svlibchecker.py --config kInduction loop-simple-safe.svlib
./svlibchecker/svlibchecker.py --config bmc loop-simple-unsafe.svlib
```

For the unsafe program it writes a trace instead:

Violation Witness

```
((select-trace
  (model) (init-global-vars (b (- 1)) (c (- 1)))
  (entry-proc main) (steps (init-proc-vars main (a (- 1))))
  (incorrect-annotation assertion :check-true false)))
```

Making Implementation easier: SvLibChecker [6]

- (a) **Nine** model-checking algorithms
- (b) Each 100 LOC on average
- (c) All algorithms together
961 LOC
- (d) Entire repository 3849 LOC
- (e) Well-documented and easy
to extend

Algorithm	Stand-Alone LOC
Predicate Abstraction [11, 13]	107
Trace Abstraction [15, 16]	105
IMC [18]	145
ISMC [19, 3]	120
IC3/PDR [9, 10]	165
BMC [8]	85
<i>k</i> -Induction [12, 4]	61
Value Analysis [7]	75
Symbolic Execution [17]	98
All Algorithms	961

Witnesses Travel Between Tools

bash

```
# CPAChecker proves the task and writes its argument
cpachecker loop-simple-safe.svlib

# PySvLib validates that argument
pysvlib validate --solver z3 --witness output/witness.svlib \
  loop-simple-safe.svlib # correct

# SvLibChecker re-uses it: 2 instead of 7 interpolation queries
./svlibchecker/svlibchecker.py --config predicateAnalysisCegar \
  --witness output/witness.svlib loop-simple-safe.svlib
```

No tool knows about any other tool — they only share the format

From Btor2 to SV-LIB

bash

```
pysvlib btor2_to_svlib bitvector-counter.btor2 --output-dir output
```

```
1 sort bitvec 4
2 zero 1
3 state 1 counter
4 init 1 3 2
5 one 1
6 add 1 3 5
7 next 1 3 6
8 ones 1
9 sort bitvec 1
10 eq 9 3 8
11 bad 10

(set-logic QF_AUFBV)

(define-proc btor2_proc () ()
  ((counter (_ BitVec 4)) (op6 (_ BitVec 4))
   (counter_next7 (_ BitVec 4)) (op10 (_ BitVec 1)))
  (! (sequence
    (assign (counter (_ bv0 4)))
    (! (while true
      (sequence
        (assign (op6 (bvadd counter (_ bv1 4))))
        (assign (op10 (bvcomp counter (_ bv15 4))))
        (! (sequence) :tag bad_state_11
          :check-true (not (= op10 (_ bv1 1))))
        (assign (counter_next7 op6))
        (assign (counter counter_next7))))
      :tag btor2_proc_while_loop))
    :tag btor2_proc_body))

(verify-call btor2_proc ())
```

From MoXI to SV-LIB

bash

```
pysvlib moxi_to_svlib integer-counter.moxi --output-dir output
```

```
(set-logic QF_LIA)

(define-system counter
  :input ()
  :output ((c Int))
  :local ()
  :init (= c 0)
  :trans (= c' (+ c 1))
  :inv true)

(check-system counter
  :input ()
  :output ((c Int))
  :local ()
  :reachable (negative (< c 0))
  :query (check-negative (negative)))

; the step: current state to next state
(define-proc counter ((c Int)) ((|c'| Int)) ()
  (! (sequence (havoc |c'|) (assume (= |c'| (+ c 1)))))
  :tag counter_moxi-proc-body))

; the query: the step, run in a loop
(define-proc counter_transition_system_check-negative
  () ((c Int)) ((|c'| Int))
  (! (sequence
    (havoc c)
    (assume (= c 0))
    (! (while true
      (sequence
        (assume true)
        (! (sequence) :tag ..._reachable
          :check-true (not (< c 0)))
        (call counter (c) (|c'|))
        (assign (c |c'|))))
      :tag ..._while_true_loop))
    :tag ..._body)))
```

From LLVM to SV-LIB

bash

```
pysvlib llvm_to_svlib add.ll --output-dir output
```

```
define i32 @add(i32 %x, i32 %y) {  
entry:  
    %sum = add i32 %x, %y  
    ret i32 %sum  
}
```

```
(define-proc file0_add  
  ((x (_ BitVec 32)) (y (_ BitVec 32)))  
  ((|__llvm_translated_retval__file0_add| (_ BitVec 32)))  
  ((sum (_ BitVec 32)))  
  (! (sequence  
    (! (label entry) :tag proc-file0_add-entry)  
    (assign (sum (bvadd x y)))  
    (assign (|__llvm_translated_retval__file0_add| sum))  
    (return))  
    :tag proc-file0_add))
```

A program, but no specification — LLVM has none to translate

Specifications Come Separately

```
(set-logic QF_BV)

(annotate-tag
  proc-file0_add
  :ensures (= |__llvm_translated_retval__file0_add| (bvadd x y)))

(declare-const a (_ BitVec 32))
(declare-const b (_ BitVec 32))

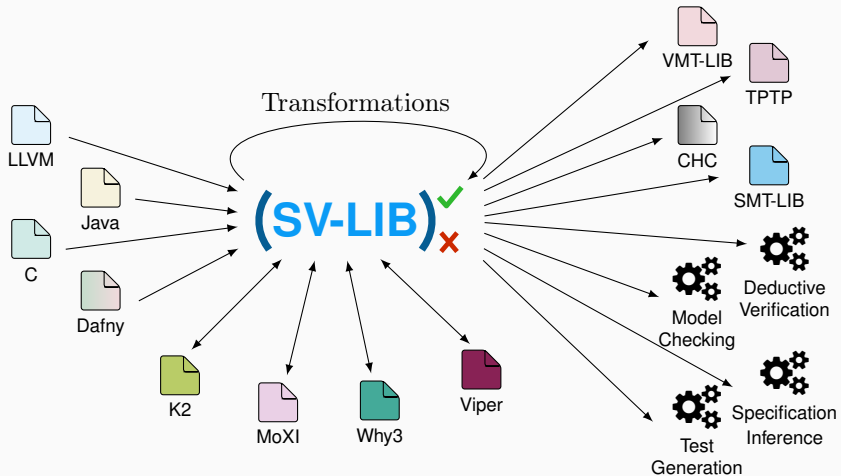
(verify-call file0_add (a b))
```

Written against the translated program, in a file of its own: the translation is never touched again when the specification changes.

bash

```
./svlibchecker/svlibchecker.py --config predicateAnalysisCegar \  
  output/translated-program.svlib add-specification.svlib # correct
```

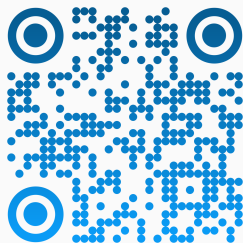
Future Work



Conclusion

- (a) SV-LIB 1.0 format available [5]!
- (b) Reduces engineering complexity for static analysis tools
- (c) First-class support for witnesses and validation
- (d) Makes information exchange between tools easier (e.g., witnesses, ...)
- (e) Tool support available — try it out yourself!

<https://gitlab.com/sosy-lab/research/data/sv-lib-demo>



<https://gitlab.com/sosy-lab/benchmarking/sv-lib>

References i

- [1] Baier, D., Beyer, D., Chien, P.C., Jakobs, M.C., Jankola, M., Kettl, M., Lee, N.Z., Lemberger, T., Lingsch-Rosenfeld, M., Wachowitz, H., Wendler, P.: Software verification with CPACHECKER 3.0: Tutorial and user guide. In: Proc. FM. pp. 543–570. LNCS 14934, Springer (2024). https://doi.org/10.1007/978-3-031-71177-0_30
- [2] Barrett, C., Stump, A., Tinelli, C.: The SMT-LIB Standard: Version 2.0. Tech. rep., University of Iowa (2010), <https://smtlib.cs.uiowa.edu/papers/smt-lib-reference-v2.0-r10.12.21.pdf>
- [3] Beyer, D., Chien, P.C., Jankola, M., Lee, N.Z.: A transferability study of interpolation-based hardware model checking for software verification. Proc. ACM Softw. Eng. **1**(FSE) (2024). <https://doi.org/10.1145/3660797>

References ii

- [4] Beyer, D., Dangl, M., Wendler, P.: Boosting k-induction with continuously-refined invariants. In: Proc. CAV. pp. 622–640. LNCS 9206, Springer (2015). https://doi.org/10.1007/978-3-319-21690-4_42
- [5] Beyer, D., Ernst, G., Jonáš, M., Lingsch-Rosenfeld, M.: SV-LIB 1.0: A standard exchange format for software-verification tasks. arXiv/CoRR **2511**(21509) (December 2025).
<https://doi.org/10.48550/arXiv.2511.21509>
- [6] Beyer, D., Lingsch-Rosenfeld, M.: SVLIBCHECKER: A light-weight tool for software model checking. In: Proc. CAV, part 3. pp. 244–260. Springer (2026). https://doi.org/10.1007/978-3-032-32537-2_12

- [7] Beyer, D., Löwe, S.: Explicit-state software model checking based on CEGAR and interpolation. In: Proc. FASE. pp. 146–162. LNCS 7793, Springer (2013). https://doi.org/10.1007/978-3-642-37057-1_11
- [8] Biere, A., Cimatti, A., Clarke, E.M., Zhu, Y.: Symbolic model checking without BDDs. In: Proc. TACAS. pp. 193–207. LNCS 1579, Springer (1999). https://doi.org/10.1007/3-540-49059-0_14
- [9] Bradley, A.R.: SAT-based model checking without unrolling. In: Proc. VMCAI. pp. 70–87. LNCS 6538, Springer (2011). https://doi.org/10.1007/978-3-642-18275-4_7

- [10] Cimatti, A., Griggio, A.: Software model checking via IC3. In: Proc. CAV. pp. 277–293. LNCS 7358, Springer (2012).
https://doi.org/10.1007/978-3-642-31424-7_23
- [11] Clarke, E.M., Grumberg, O., Jha, S., Lu, Y., Veith, H.: Counterexample-guided abstraction refinement. In: Proc. CAV. pp. 154–169. LNCS 1855, Springer (2000). https://doi.org/10.1007/10722167_15
- [12] Donaldson, A.F., Haller, L., Kröning, D., Rümmer, P.: Software verification using *k*-induction. In: Proc. SAS. pp. 351–368. LNCS 6887, Springer (2011).
https://doi.org/10.1007/978-3-642-23702-7_26

- [13] Flanagan, C., Qadeer, S.: Predicate abstraction for software verification. In: Proc. POPL. pp. 191–202. ACM (2002).
<https://doi.org/10.1145/503272.503291>
- [14] Griggio, A., Jonáš, M.: KRATOS2: An SMT-based model checker for imperative programs. In: Proc. CAV. pp. 423–436. Springer (2023).
https://doi.org/10.1007/978-3-031-37709-9_20
- [15] Heizmann, M., Hoenicke, J., Podelski, A.: Refinement of trace abstraction. In: Proc. SAS. pp. 69–85. LNCS 5673, Springer (2009).
https://doi.org/10.1007/978-3-642-03237-0_7

- [16] Heizmann, M., Hoenicke, J., Podelski, A.: Software model checking for people who love automata. In: Proc. CAV. pp. 36–52. LNCS 8044, Springer (2013). https://doi.org/10.1007/978-3-642-39799-8_2
- [17] King, J.C.: Symbolic execution and program testing. Commun. ACM **19**(7), 385–394 (1976). <https://doi.org/10.1145/360248.360252>
- [18] McMillan, K.L.: Interpolation and SAT-based model checking. In: Proc. CAV. pp. 1–13. LNCS 2725, Springer (2003). https://doi.org/10.1007/978-3-540-45069-6_1
- [19] Vizel, Y., Grumberg, O.: Interpolation-sequence based model checking. In: Proc. FMCAD. pp. 1–8. IEEE (2009). <https://doi.org/10.1109/FMCAD.2009.5351148>